"بسم الله الرحمن الرحيم"

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5341, Machine Learning and Data Science

Assignment 1

Data Preprocessing & Exploratory Data Analysis (EDA)

Prepared by:

Khaled Hammad	**ID: 1220857**	**Sec: 1**
Mohammad Shamasneh	**ID: 1220092**	**Sec: 1**

Instructor: Dr. Yazan Abu Farha

Date of submission: 10/23/2025

Table of Contents

Tasks 2
T1: Data Loading and Initial Inspection 2
T2: Handling Missing Data 4
T3: Handling Outliers 6
T4: Feature Scaling 9
T5: Exploratory Data Analysis (EDA) 10
1. Univariate Analysis 10
2. Bivariate Analysis 12
3. Correlation Analysis 14
T6: Data Visualizations 17
Conclusion 19
References: 20

Tasks

T1: Data Loading and Initial Inspection

The first part of the code begins by importing several essential Python libraries used for data analysis and visualization

```
import pandas
import seaborn
import matplotlib
from sklearn.preprocessing import StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.pyplot import title
```

Figure 1: Python Libraries

where import pandas for handling data, , from sklearn.preprocessing import StandardScaler for data normalization, import matplotlib.pyplot as plt and import seaborn as sns and from matplotlib.pyplot import title for plotting

```
##############################################################################
#############################--|  task 1  |---#####################################
##############################################################################

data = pandas.read_csv("customer_data.csv")


##############################################################################
# ------------| to show if theis non binary value at binary feature  |--------------

# for col in ['Gender', 'ProductType', 'ChurnStatus']:
#     unique_vals = data[col].unique()
#     print(f"Unique values in column '{col}': {unique_vals}")
#
#     if set(unique_vals).issubset({0, 1}):
#         print(f"column '{col}' contains only 0 and 1")
#     else:
#         print(f"column '{col}' contains values other than 0 and 1")

##############################################################################
```

Figure 2: Read Dataset

We use the function pandas.read_csv(“ ”) to read the data from **customer_data.csv** file, This function loads the data into a **DataFrame**, which is a core data structure in the pandas library, similar to table, where the loaded data is stored in the variable data .

Then verify whether category columns (Gender, ProductType, and ChurnStatus) contain only binary values (0 and 1). This check is important to ensure data consistency for features that are expected to be binary, we commented out after the first check.

```python

print(data.head())
print("------------------------------------------------------------")
print(data.info())
print("------------------------------------------------------------")
print(data.describe())
print("------------------------------------------------------------")

```

Figure 3: Get some information about data set

At this part, we inspect the first five rows of the dataset using **.head( ),** check the general information **.info( )** and Generate summary statistics **.describe( ).**

```
PS C:\Users\moham\OneDrive\Desktop\ML\assignment_1_solve> python -u "c:\Users\moham\OneDrive\Desktop\ML\assignment_1_solve\slove.py"
  CustomerID   Age  Gender    Income  Tenure  ProductType  SupportCalls  ChurnStatus
0   CUST0000  59.0       0  151203.0     4.0            0           1.0            0
1   CUST0001  69.0       0   58332.0     6.0            1           9.0            0
2   CUST0002  46.0       1  149481.0     2.0            0          12.0            0
3   CUST0003  32.0       1  115937.0     1.0            1          13.0            0
4   CUST0004  60.0       0  103929.0     4.0            1           5.0            0
------------------------------------------------------------
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3500 entries, 0 to 3499
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   CustomerID    3500 non-null   object
 1   Age           3325 non-null   float64
 2   Gender        3500 non-null   int64
 3   Income        3328 non-null   float64
 4   Tenure        3325 non-null   float64
 5   ProductType   3500 non-null   int64
 6   SupportCalls  3329 non-null   float64
 7   ChurnStatus   3500 non-null   int64
dtypes: float64(4), int64(3), object(1)
memory usage: 218.9+ KB
None
------------------------------------------------------------
               Age       Gender        Income       Tenure  ProductType  SupportCalls  ChurnStatus
count  3325.000000  3500.000000  3.328000e+03  3325.000000  3500.000000   3329.000000  3500.000000
mean     43.606617     0.495714  1.406860e+05     5.041504     0.298857     10.078702     0.044857
std      14.931487     0.500053  4.333271e+05     2.571029     0.457822     21.735374     0.207020
min      14.000000     0.000000  2.503700e+04     0.000000     0.000000      1.000000     0.000000
25%      31.000000     0.000000  5.653025e+04     3.000000     0.000000      3.000000     0.000000
50%      43.000000     0.000000  8.953250e+04     5.000000     0.000000      7.000000     0.000000
75%      56.000000     1.000000  1.215025e+05     7.000000     1.000000     11.000000     0.000000
max      69.000000     1.000000  5.004849e+06     9.000000     1.000000    200.000000     1.000000
---------------------| done task 1 |-----------------------------------
```

Figure 4: Information about data set

As we can observe, calling **.head()** prints the first five rows of the dataset. This gives us a quick overview of the data, allowing us to examine the initial records. On the other hand, **.info()** provides general information about the dataset, such as the type of data structure (a pandas **DataFrame**), the index range (from 0 to 3499 in this case), the total number of columns, and for each column, the number of non-null values and its data type. This method is helpful for identifying missing data and understanding the overall structure of the dataset,

the **.describe()** method generates summary statistics for the numeric columns in the dataset, including **count**, **mean**, **Q1** (25th percentile), **Q2** (50th percentile or median), **Q3** (75th percentile), **min**, **max**, and **standard deviation**. Useful for quickly understanding the distribution and range of numeric data.

T2: Handling Missing Data

Sometimes data may be in the right format, but some of the values are missing may be due to problems with the process of collecting data, or an equipment malfunction. Or, comprehensiveness may not have been considered important at the time of collection. Furthermore, some data may get lost due to system or human error while storing or transferring the data [1].

Strategies to handle missing data:

• Use a measure of central tendency for the attribute (e.g., the mean or median) to fill in the missing value: For normal (symmetric) data distributions, the mean can be used, while skewed data distribution should employ the median. We will talk about skewed distribution later in the lecture [1].

We use median attributes to fill the missing value.

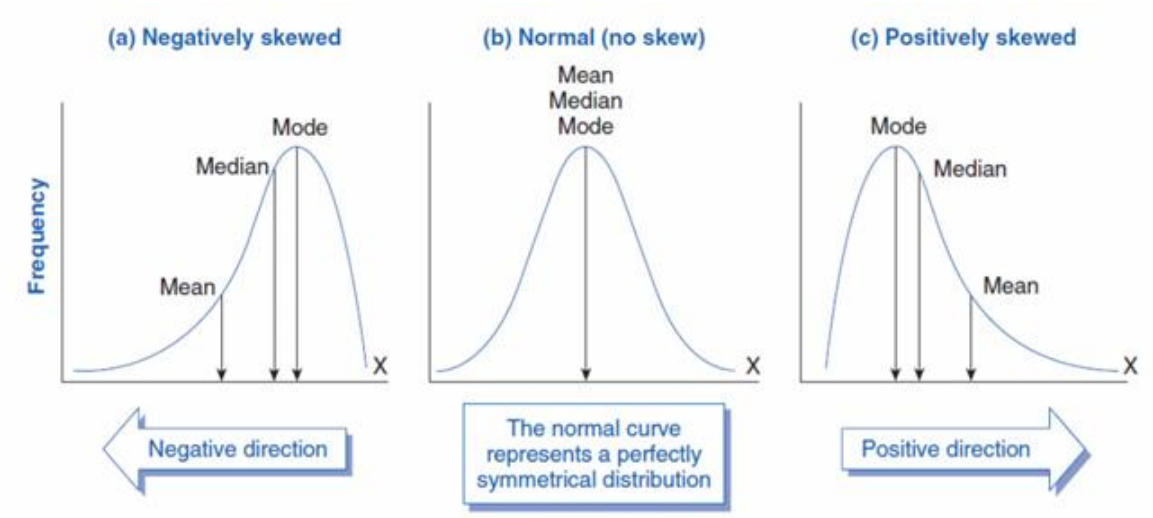

Figure 5: Mean vs Median vs Mode

Solve

As shown in the code, we use **.isnull().sum()** to calculate the number of missing values in each column. This helps us identify which columns contain missing data. Next, we handle

the missing values by filling them with **the median**, especially since the data **contains outliers** that could skew other methods like the mean.

For the missing values in the Income column, we rely on the **ProductType** column to fill the gaps. We group the data by **ProductType** (which has two categories: **Basic** and **Premium**) and compute the median Income for each group. Then, for each missing value in the **Income** column, we fill it with the appropriate median based on the product type: if the product type is Basic, we use the median income of the Basic group, and if the product type is 'Premium', we use the median of the Premium group.

```
##############################################################################
#############################--|  task 2  |---##################################
##############################################################################

print(data.isnull().sum())

data['Age'] = data['Age'].fillna(data['Age'].median())
data['Tenure'] = data['Tenure'].fillna(data['Tenure'].median())
data['SupportCalls'] = data['SupportCalls'].fillna(data['SupportCalls'].median())
data['Income'] = data.groupby('ProductType')['Income'].transform(lambda x: x.fillna(x.median()))

##############################################################################
#############################--|  task 3  |---##################################
##############################################################################
```

Figure 6: Handle Missing value

The number of missing value for each columns:

```
CustomerID        0
Age             175
Gender            0
Income          172
Tenure          175
ProductType       0
SupportCalls    171
ChurnStatus       0
dtype: int64
```

Figure 7: Number of missing value

T3: Handling Outliers

The quartiles of a population or a sample are the three values which divide the distribution or observed data into even fourths.

- The first quartile, Q1, is the value for which 25% of the observations are smaller and 75% are larger
- Q2 is the same as the median (50% are smaller, 50% are larger)
- Only 25% of the observations are greater than the third quartile [1].

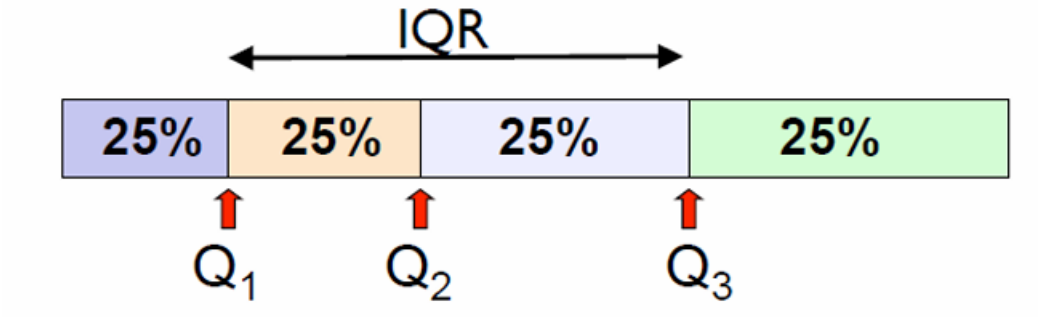

Figure 8: IQR [1].

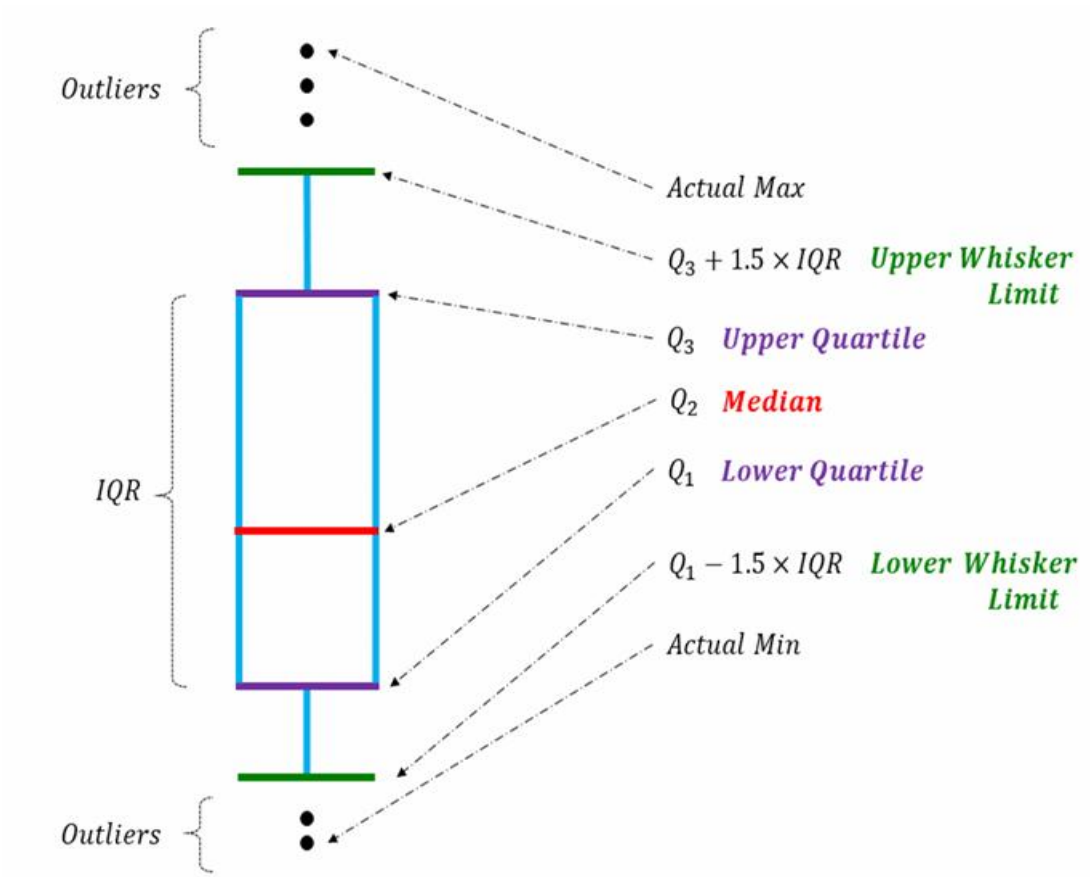

Figure 9: Box Plot [1].

Solve

After we handle a missing data in task 2, we want now to handle outlier. Before we handle outlier, we will see the data using box plots as we shown in the figure below.

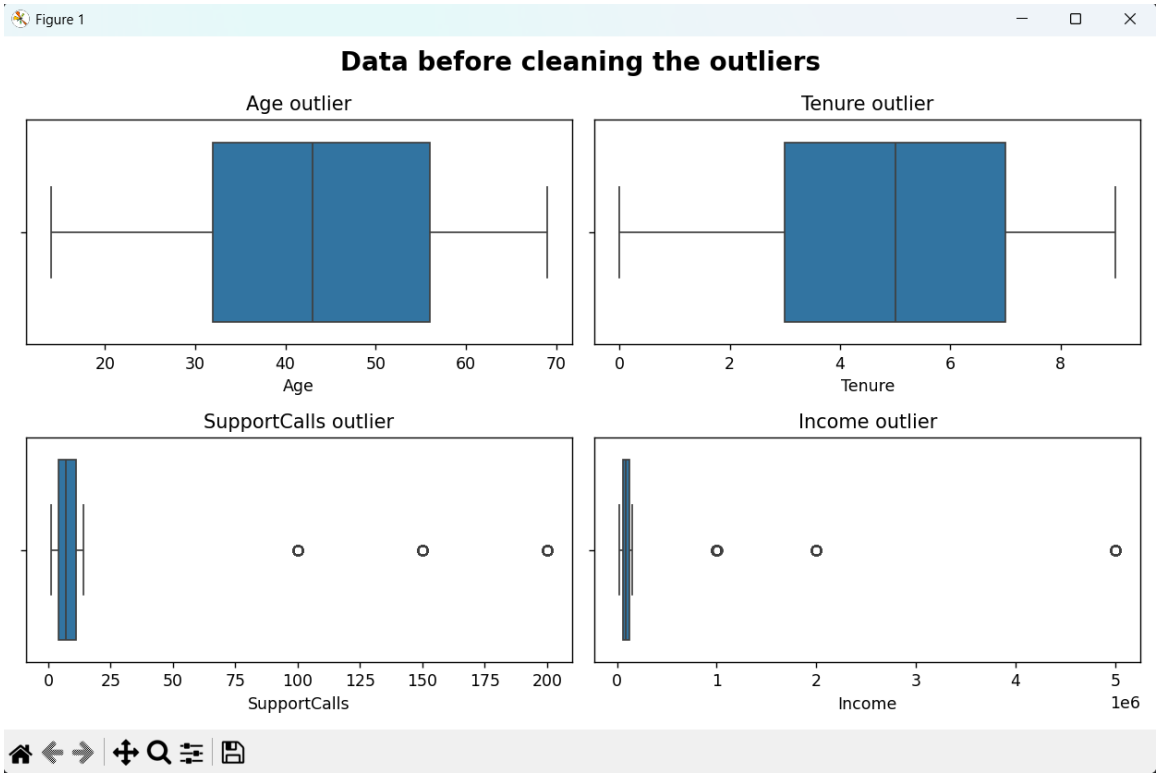

Figure 10: Data before cleaning outliers.

As we see above, they have an outlier data in **supportcalls** and **Income** so we must handle it as we seen below.

```
for col in ['Age', 'Tenure', 'SupportCalls','Income']:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3- Q1
    lowerBound = Q1 - 1.5 * IQR
    upperBound = Q3 + 1.5 * IQR
    medianValue = data[col].median()
    data[col] =data[col].apply(lambda x: medianValue if x < lowerBound or x > upperBound else x)
```

Figure 11: Handling outliers Code.

As we shown in the code, I handle the outlier by change its variable not by ignore it I see that this is the best because we suppose that there is an important data in file and if we want to ignore each data, they have outlier we will lose this data that its important.

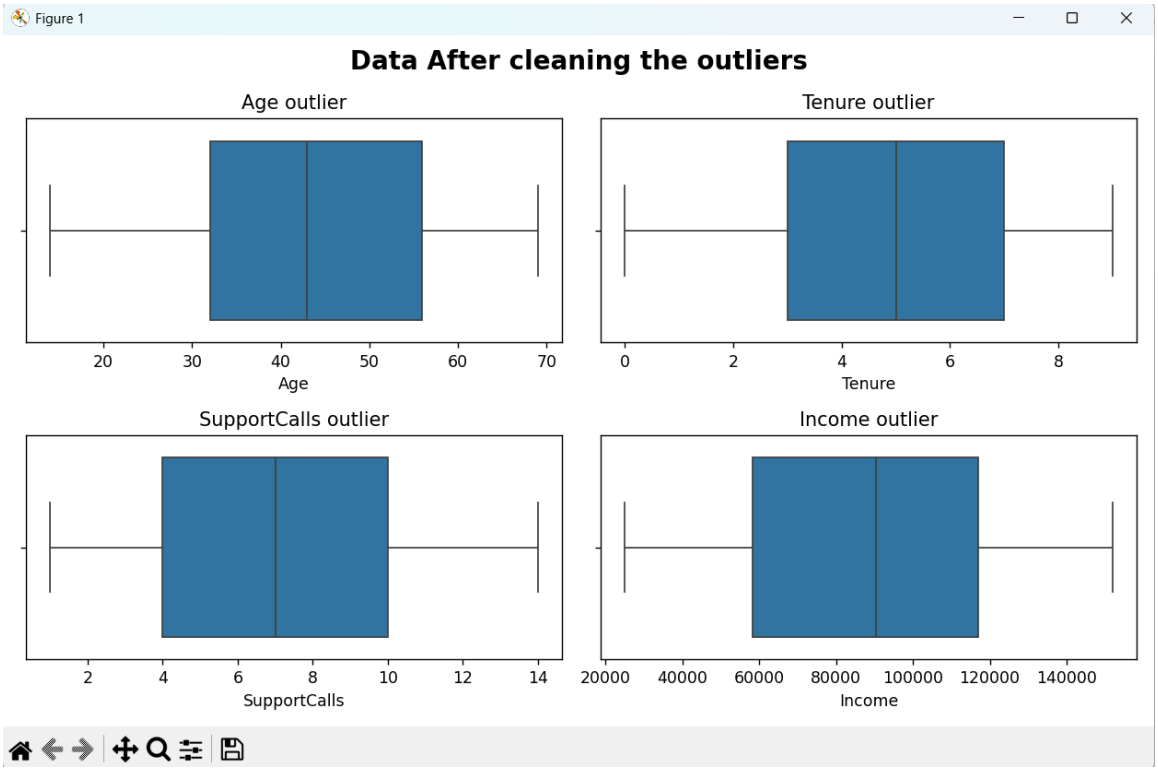

Figure 12: Data after cleaning outliers.

As we shown above the figure display the data after cleaning it from outliers.

T4: Feature Scaling

Since the range of values of raw data varies widely, in some machine learning algorithms, objective functions will not work properly without normalization. Therefore, the range of all features should be normalized so that each feature contributes approximately proportionately to the final distance [1].

Min-Max Normalization:

$$v' = \frac{v - min_A}{max_A - min_A}(newMax_A - newMin_A) + newMin_A$$

Figure 13: Min-Max Normalization equation

- works as a linear transformation to a given range.
- In this approach, the data is scaled to a fixed range - usually 0 to 1.
- The technique keeps relationship among original data.
- The cost of having this bounded range - in contrast to standardization - is that we will end up with smaller standard deviations, which can suppress the effect of outliers [1].

Slove:

After we handle missing data and outliers' data we want now to normalized data. We know that **Min-Max** algorithm is sensitive to outliers but because we cleaned the data from outliers above, we will use it instead of using **standardization** because it will work as it so they will not be affected.

```
for col in ['Age', 'Tenure', 'SupportCalls','Income']:
    minv = data[col].min()
    maxv = data[col].max()
    data[col] = (data[col]-minv)/(maxv-minv)
```

Figure 14: Feature scaling Code.

Note that we normalized the data range between 0-1.

T5: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an essential step in the data analysis process, aimed at summarizing and visualizing the key characteristics of a dataset, often with the help of graphical representations. It helps in understanding the structure, relationships, and distributions within the data, while identifying potential outliers or anomalies. EDA employs various techniques such as histograms, box plots, and scatter plots, as well as calculating summary statistics like mean, median, and variance. By performing EDA, analysts can gain insights that guide data cleaning, transformation, and modeling, ensuring that the data is suitable for further analysis. [2]

In this task, we explore and analyze the dataset to uncover patterns, relationships, and anomalies. The following steps were taken to examine the dataset, understand its distribution, and identify relationships between variables, with a particular focus on the target variable, **ChurnStatus**.

1. Univariate Analysis

Visualizing the distribution of numerical features: **Histograms** were created for the numerical features: **Age, Tenure, SupportCalls, and Income**. These plots help visualize the distribution and spread of the data, identify any skewness, and detect any potential outliers.

```
##############################################################################
#############################--|  task 5  |---##################################
##############################################################################

fig, axes = plt.subplots(2, 2, figsize=(10, 6))
fig.suptitle('Distribution of Data', fontsize=16, fontweight='bold')
sns.histplot(x=data['Age'], ax=axes[0,0])
axes[0,0].set_title("Age distribution")
sns.histplot(x=data['Tenure'], ax=axes[0,1])
axes[0,1].set_title("Tenure distribution")
sns.histplot(x=data['SupportCalls'], ax=axes[1,0])
axes[1,0].set_title("SupportCalls distribution")
sns.histplot(x=data['Income'], ax=axes[1,1])
axes[1,1].set_title("Income distribution")
plt.tight_layout()
plt.show()

##############################################################################

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('Distribution of Categories', fontsize=16, fontweight='bold')
sns.countplot(x=data['Gender'], ax=axes[0, 0])
axes[0, 0].set_title("Gender Distribution")
sns.countplot(x=data['ProductType'], ax=axes[0, 1])
axes[0, 1].set_title("Product Type Distribution")
sns.countplot(x=data['ChurnStatus'], ax=axes[1, 0])
axes[1, 0].set_title("Churn Status Distribution")
axes[1, 1].axis('off')
plt.tight_layout()
plt.show()
```

Figure 15: Univariate Analysis

As we see at the figure 16 below, the distribution of numerical data, we see the data with no outliers and with normalization (min-max normalization), the Age and income has an approximately uniform distribution.

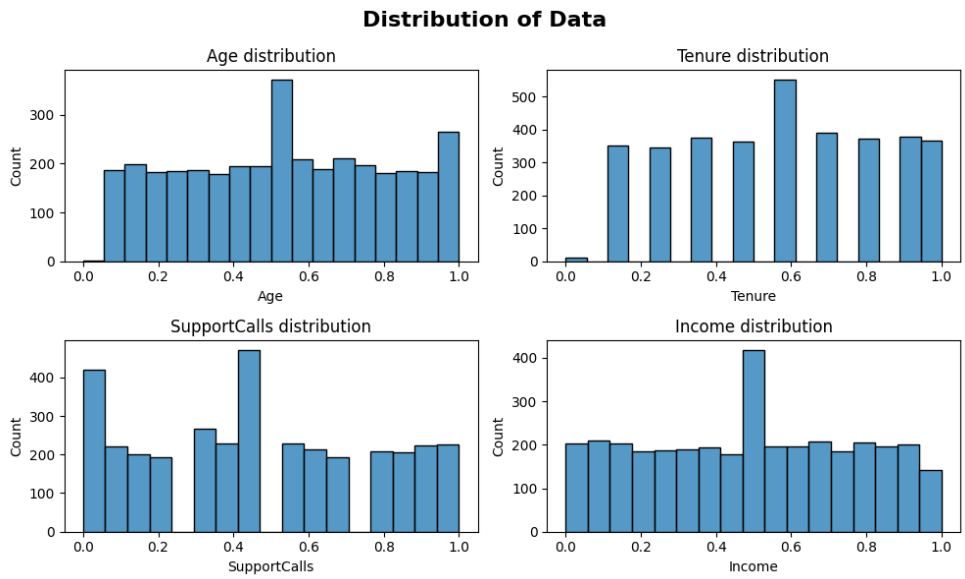

Figure 16: Distribution of numerical data

Visualizing the distribution of categorical features: count plots were created for categorical variables: **Gender, ProductType, and ChurnStatus**. These help to understand the distribution of the different categories and their balance in the dataset.

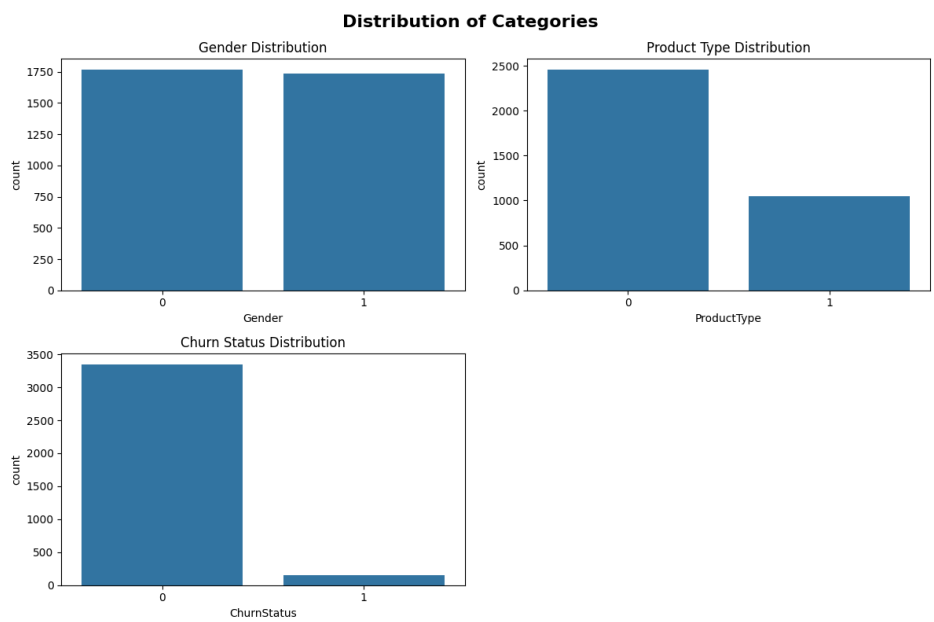

Figure 17: Distribution of categories data

2. Bivariate Analysis

Numerical features vs. ChurnStatus: **Box plots** were used to explore the relationship between numerical features and the target variable ChurnStatus. This allows us to analyze how the distributions of Income, **Age, SupportCalls, and Tenure** vary between churned and non-churned customers.

```
#  relationships between numerical features and the target variable using or box plots.

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('Distribution by Churn Status', fontsize=16, fontweight='bold')
sns.boxplot(x=data['ChurnStatus'], y=data['Income'], ax=axes[0, 0])
axes[0, 0].set_title('Income Distribution by Churn Status')
sns.boxplot(x=data['ChurnStatus'], y=data['Age'], ax=axes[0, 1])
axes[0, 1].set_title('Age Distribution by Churn Status')
sns.boxplot(x=data['ChurnStatus'], y=data['SupportCalls'], ax=axes[1, 0])
axes[1, 0].set_title('Support Calls Distribution by Churn Status')
sns.boxplot(x=data['ChurnStatus'], y=data['Tenure'], ax=axes[1, 1])
axes[1, 1].set_title('Tenure Distribution by Churn Status')
plt.tight_layout()
plt.show()

#Investigate relationships between categorical variables and the target using bar plots

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
sns.countplot(x='Gender', hue='ChurnStatus', data=data, ax=axes[0])
axes[0].set_title('Churn Status by Gender')
sns.countplot(x='ProductType', hue='ChurnStatus', data=data, ax=axes[1])
axes[1].set_title('Churn Status by Product Type')
plt.tight_layout()
plt.show()
```

Figure 18: Bivariate Analysis

As shown in the figures 19 below, the relationship between numerical features and Churn Status becomes clearer:

- Income: Churned customers generally have lower incomes, while non-churned customers tend to have higher incomes.
- Age: There is little difference in age distribution between churned and non-churned customers, suggesting age is not a major factor in churn.
- Support Calls: Churned customers make fewer support calls than non-churned customers.
- Tenure: Non-churned customers have a wider range of tenures, while churned customers tend to have shorter tenures.

These observations provide useful insights into potential factors influencing churn.

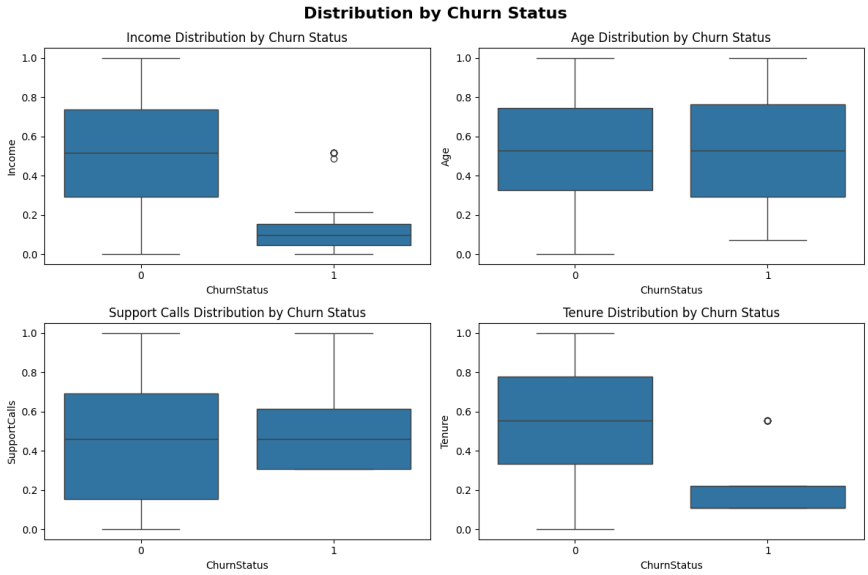

Figure 19: relationship between numerical features and Churn Status

Categorical features vs. ChurnStatus:Count plots were used to examine the relationship between categorical variables (Gender and ProductType) and the target variable ChurnStatus. These plots allow us to assess how the distribution of ChurnStatus varies by gender and product type.

From the figure 20, we can observe that Gender does not show a significant impact on the churn status, as the number of customers who stayed and those who churned is almost equal for both genders.

Regarding Product Type, the Basic product appears to be more common than the Premium one, with slightly higher churn counts.

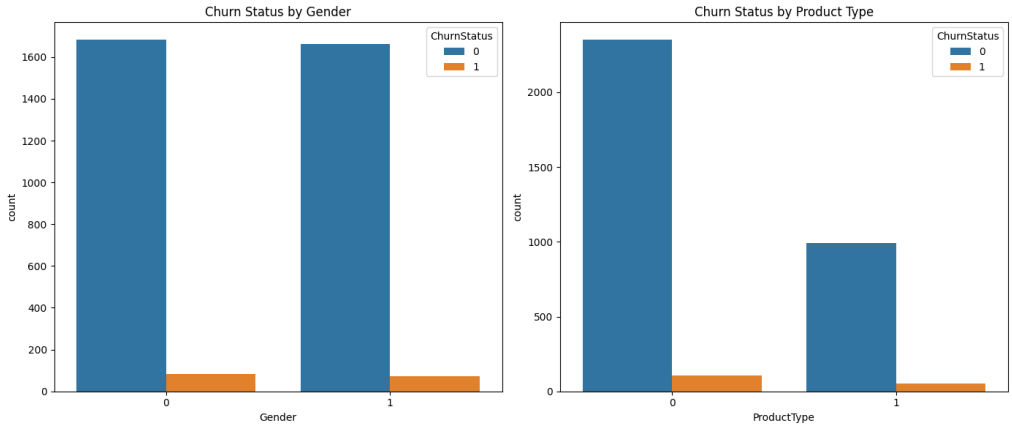

Figure 20: relationship between categorical features and Churn Status

Additional :

We make a **Triangular relationship** between two variables and the target variable, we see the relation between income and Tenure with target variable (ChurnStatus) is appear clearly where with Customers with lower Income and shorter Tenure are more likely to have ChurnStatus = 1 as they are clustered in the lower-left corner of the plot.

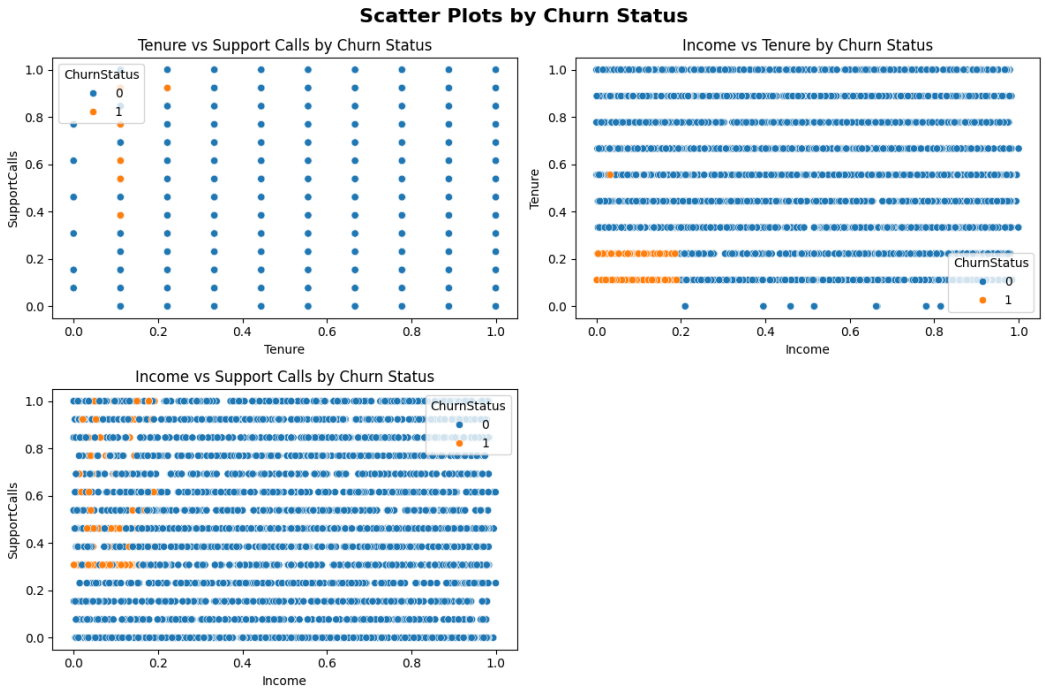

Figure 21: Scatter plots by Churn Status

3. Correlation Analysis

Correlation is a statistical analysis that is used to measure and describe the strength and direction of the relationship between two variables. [1]

• Strength indicates how closely two variables are related to each other, and direction indicates how one variable would change its value as the value of the other variable changes. [1]

• Directions: All correlation coefficients between 0 and 1 represent positive correlations, while all coefficients between 0 and -1 are negative correlations. Positive relation means if one increase (decrease), then other will increase (decrease). Negative relation means if one increase (decrease), then other will decrease (increase). [1]

• Strength: The closer a correlation coefficient is to 1 or to -1, the stronger it is. Following picture suggests a guideline to interpret the strength. [1]

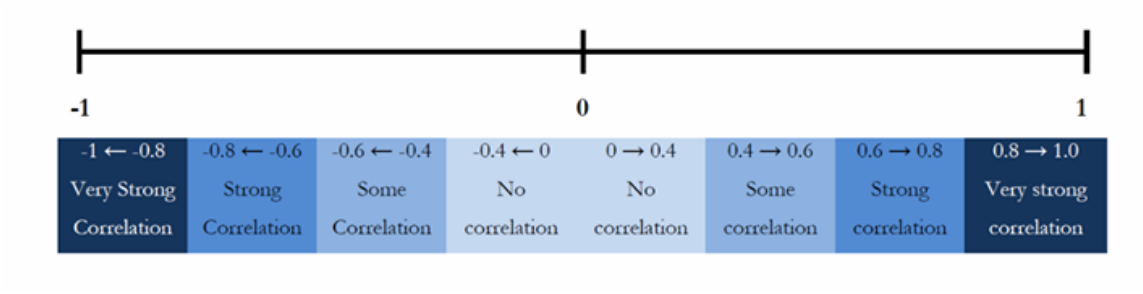

Figure 22: strength of Correlation Analysis

Correlation Matrix:

A correlation matrix was generated to analyze the relationships between numerical features. This matrix visually shows how strongly each feature correlates with others and with the target variable **ChurnStatus**. The **heatmap** provides a clear, easy-to-read view of these correlations, helping to identify which features are most predictive of churn.

```
# Correlation Analysis


correlation_matrix = data.select_dtypes(include=['number']).corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix')
plt.show()
```

Figure 23: Correlation Analysis

From the correlation matrix, we can observe how each feature relates to the target variable **ChurnStatus**:

Note: Diagonal values (1.00) Each variable is perfectly correlated with itself, so these values are always 1.

- **Income (-0.30)** : Shows a moderate negative correlation with churn, meaning that customers with higher income are less likely to leave the company.
- **Tenure (-0.30)** : Also has a moderate negative correlation, indicating that customers who have stayed longer are less likely to churn.
- **SupportCalls (0.03)** : Has a very weak positive correlation, suggesting that the number of support calls has almost no effect on churn behavior.
- **Age (-0.00)** and **Gender (-0.01)** : Both show no significant correlation with churn, implying that these factors do not influence whether a customer leaves.
- **ProductType (0.02)** : Displays a very small positive correlation, meaning product type has minimal impact on churn.

As the figure 22 shows, there is no significant correlation (values between 0 and 0.4 are generally considered weak). However, we will consider **Income** and **Tenure** as having a moderate relationship for analysis purposes. Although the correlation between **Income** and **ChurnStatus** is weak (approximately −0.03), this may be due to the small number of churned customers in the dataset. The limited churn data reduces statistical power, so a potential relationship between lower income and higher churn likelihood cannot be ruled out.

This weak correlation is also influenced by the binary nature of the target variable (ChurnStatus = 0 or 1**)**, which restricts the variability that correlation can capture. As shown in the figure, the customers who churned are clustered in the corner with lower values of Income and Tenure, indicating that customers with lower income and shorter tenure tend to have a higher likelihood of churn.

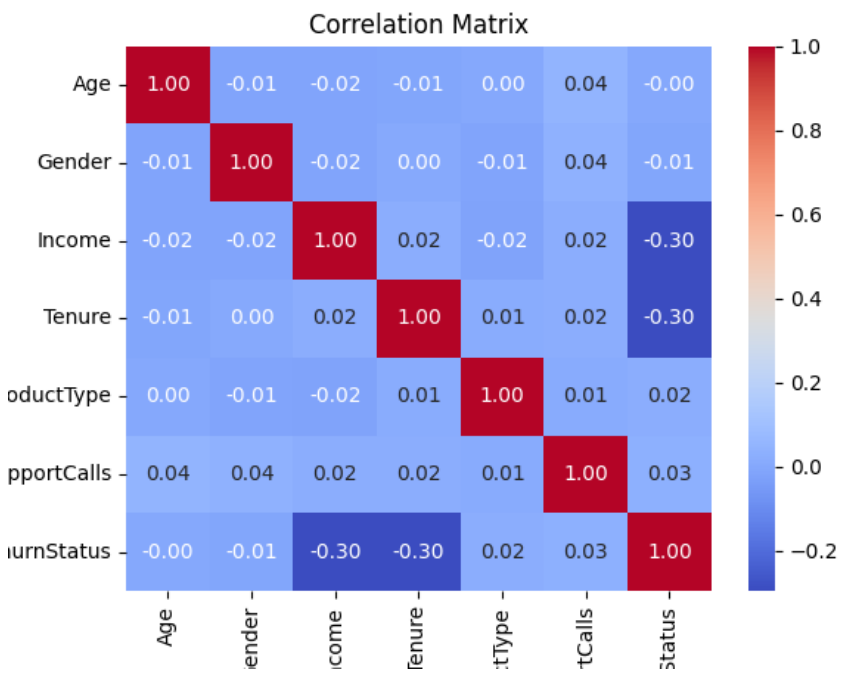

Figure 24: Correlation matrix

T6: Data Visualizations

What is data visualization

• Data visualization means encoding information about the data in a graphical way. [1]
• It helps to highlight the most useful insights from a dataset, making it easier to spot trends, patterns, outliers, and correlations. [1]
• The ultimate goal of data visualization is to tell a story. We are trying to convey information about the data as efficiently as possible. [1]
• Using graphics allows the reader to quickly and accurately understand the message we are trying to transmit. [1]

Benefits of effective data visualization

• Get an initial understanding of your data by making trends, patterns, and outliers easily visible to the naked eye. [1]
• Comprehend large volumes of data quickly and efficiently. [1]
• Communicate insights and findings to non-data experts, making your data accessible and actionable. [1]
• Tell a meaningful and impactful story, highlighting only the most relevant information for a given context. [1]

We use **boxplot, boxenplot, swarmplot and histogram** plot to communicate insights from the data, and Shows relationships.

```
##########################################################################################
##################################|  task 6  |#############################################
##########################################################################################

fig, axs = plt.subplots(2, 2, figsize=(12, 10))
sns.boxplot(x='ChurnStatus', y='Tenure', data=data, ax=axs[0, 0])
axs[0, 0].set_title('Tenure vs Churn Status (Box Plot)')
sns.boxenplot(x='ChurnStatus', y='Income', data=data, ax=axs[0, 1])
axs[0, 1].set_title('Income vs Churn Status (Boxen Plot)')
sns.swarmplot(x='ChurnStatus', y='Income', data=data, ax=axs[1, 0])
axs[1, 0].set_title('Income vs Churn Status (Swarm Plot)')
sns.histplot(x='Tenure', hue='ChurnStatus', data=data, ax=axs[1, 1], kde=True)
axs[1, 1].set_title('Tenure vs Churn Status (Hist Plot)')
plt.tight_layout()
plt.show()


sns.scatterplot(x=data['Income'], y=data['Tenure'], hue=data['ChurnStatus'])
plt.title('Income vs Tenure by Churn Status')
plt.show()
```

Figure 25: data visualization

The visualizations reveal a clear relationship between churn status, tenure, and income. Customers who did churn generally have the lowest tenure and lowest income levels, indicating that they tend to leave the company early and often belong to lower-income segments. In contrast, customers with longer tenure and higher income are less likely to

churn, suggesting greater satisfaction or loyalty. This pattern highlights the importance of focusing retention strategies on newer and lower-income customers to reduce churn and improve overall customer stability.

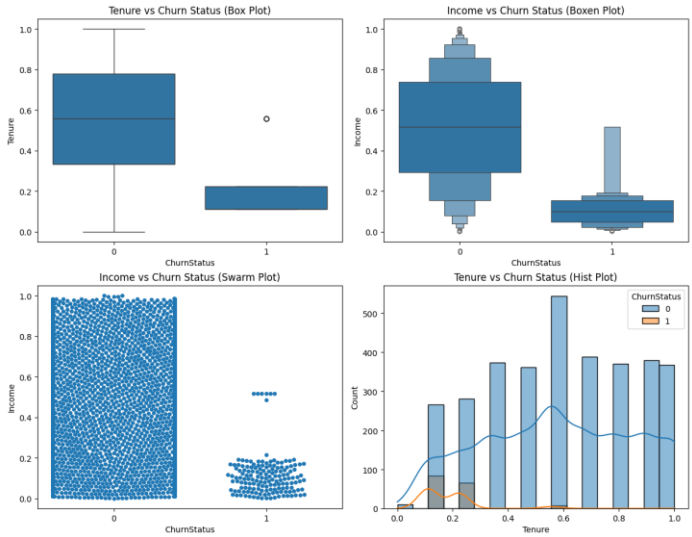

Figure 26: relationship between churn status, tenure, and income

The scatter plot Income vs Tenure by Churn Status at Figure shows the relationship between customers' income and tenure, categorized by churn status. The plot indicates that customers who churned (orange points) generally have low income and short tenure, meaning they are newer customers with lower financial stability. In contrast, customers who did not churn (blue points) are spread across higher income and tenure values, suggesting they tend to stay longer and likely have stronger engagement with the company. This relationship highlights that low-income, short-tenure customers are more prone to churn.

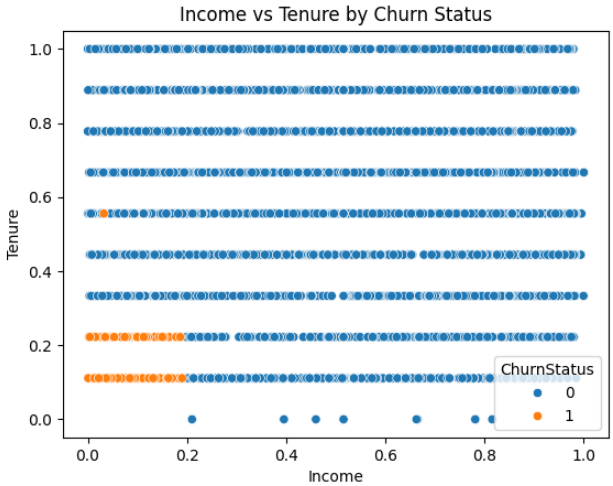

Figure 27: The scatter plot Income vs Tenure by Churn Status

Conclusion

Throughout this project, we gained valuable experience in **data science** concepts and learned how to use important **Python libraries** related to data analysis, such as **Pandas**, **Matplotlib**, and **Seaborn**. We explored how to handle and check for **missing data**, **detect and manage outliers**, and apply **normalization techniques** to prepare data for analysis. Through the process of **data preprocessing** and **exploratory data analysis (EDA)**, we identified key insights about the factors influencing customer churn. After cleaning and normalizing the data, the analysis revealed that **income** and **tenure** have a significant relationship with **churn status**. Customers with **lower income levels** and **shorter tenure** are more likely to churn, indicating that financial instability and limited engagement duration contribute to customer loss. In contrast, customers with **higher income** and **longer tenure** tend to remain loyal, showing greater satisfaction and retention. **The visualizations** effectively illustrated these relationships, providing a clear and intuitive way for both technical and non-technical audiences to understand the results.

References:

[1]: 3_data_science_intro.pdf

[2]: geeksforgeeks, What is Exploratory Data Analysis? , Click Here